

BRAICE — Colosseum Hackathon Engineering Build Plan

1. Purpose of This Document

This document defines the concrete engineering plan for building the BRAICE prototype for the Colosseum hackathon.

The objective is not to build the complete BRAICE platform described in the long-term product vision.

The objective is to build a working, demonstrable vertical slice that proves BRAICE's central technical thesis:

Community governance can be translated into machine-enforceable, revocable permissions that control how applications and AI access community-generated data.

The prototype must demonstrate this entire chain:

Community activity → Individual data → Aggregation → Access request → Governance decision → Permission → Authorized AI analysis → Revocation → Access prevented

This is consistent with the EXIST MVP definition, which explicitly states that the project should prove the complete transaction rather than attempt to build the entire platform.

2. What We Are Actually Building

The hackathon product will be called:

BRAICE Governed Community Intelligence

The prototype represents one creator-led community.

For the demo:

Community: Afrobeat Creators

Community operator: Creator

Commercial partner: Brand

Dataset: Aggregated community interests

Application: BRAICE AI Intelligence Agent

Governance rule: Creator approval + community approval threshold

The commercial partner wants to understand the community's emerging interests.

The partner does NOT receive individual member records.

Instead:

1. Community members generate activity.
2. BRAICE collects synthetic individual-level activity.
3. BRAICE aggregates the activity into community intelligence.
4. The brand submits an access request.
5. BRAICE evaluates the governance rule.
6. An approved request produces a machine-readable permission.
7. The AI agent receives only the authorized aggregated dataset.
8. The AI produces an insight.
9. The community revokes the permission.
10. The same access attempt is made again.
11. BRAICE rejects it.

That is the complete product.

3. What the Judges Should Understand in 60 Seconds

The prototype should make one thing obvious:

Traditional architecture:

```
Application
  ↓
Database
  ↓
Data
```

BRAICE architecture:

```
Application
  ↓
Permission Check
  ↓
Governance-derived Authorization
  ↓
Authorized Data
```

The fundamental difference is that **data access is no longer simply a technical ACL configured by an administrator.**

BRAICE connects:

rights → governance → permissions → enforcement

The full product concept explicitly separates data, rights, permissions and governance rather than treating them as the same thing.

4. The MVP Boundary

Build

- Community
- Members
- Individual activity records
- Aggregation pipeline
- Community-level dataset
- Access requests
- Governance rules
- Governance decisions
- Permission objects
- Permission evaluation
- AI agent
- Permission-aware AI tools
- Revocation
- Audit trail
- Solana-based verifiable governance/permission events
- Demo UI
- Automated authorization/security tests

Do NOT build

- Full social network
- Full creator platform
- Marketplace
- NFT system
- Token economy
- DAO framework
- Multi-chain support

- Multiple blockchains
- Full CRM
- General-purpose analytics
- General-purpose identity provider
- Real-world personal data collection
- Complex community voting infrastructure
- Full commercial billing system
- Portable community state across arbitrary applications

Those are future-stage capabilities.

The full BRAICE concept describes portable community identity, membership, rights, governance, permissions and applications as a long-term direction, not as requirements for the initial prototype.

5. Recommended Technology Stack

The stack should optimize for:

1. Fast development
2. Clear architecture
3. Strong security boundaries
4. Easy demonstration
5. Solana/Colosseum relevance
6. Ability to evolve after the hackathon

Frontend

Next.js + TypeScript

Use:

- Next.js
- React
- TypeScript
- Tailwind CSS
- shadcn/ui

The frontend should be a thin client.

It should NOT contain authorization logic.

The browser can request an action, but the backend must determine whether the action is permitted.

Backend

TypeScript + NestJS

Use NestJS for:

- REST APIs
- business logic
- authorization orchestration
- governance service
- permission engine
- AI orchestration
- blockchain integration
- audit events

Why NestJS?

Because BRAICE has several distinct domain services and we want the codebase to remain structured as the prototype grows.

Database

PostgreSQL

PostgreSQL stores:

- users
- communities
- memberships
- activity records
- aggregated datasets
- access requests
- governance rules
- governance decisions
- permissions
- audit events
- application registrations

Do not use the blockchain as the primary database.

The BRAICE product concept explicitly specifies that sensitive data remains off-chain.

Blockchain

Solana + Anchor

Use Solana for verifiable shared state.

Use Anchor for the program.

The Solana program should NOT contain:

- member activity
- personal data
- AI datasets
- raw community data

It should contain only the minimum state required to establish verifiable governance/authorization state.

AI

Use an LLM through a backend AI service.

The important architecture is:

```
graph TD;
  A[AI Agent] --> B[BRAICE Tool Layer];
  B --> C[Permission Engine];
  C --> D[Authorized Dataset];
```

NOT:

```
graph TD;
  A[AI Agent] --> B[PostgreSQL];
```

The AI must never receive unrestricted database credentials.

Infrastructure

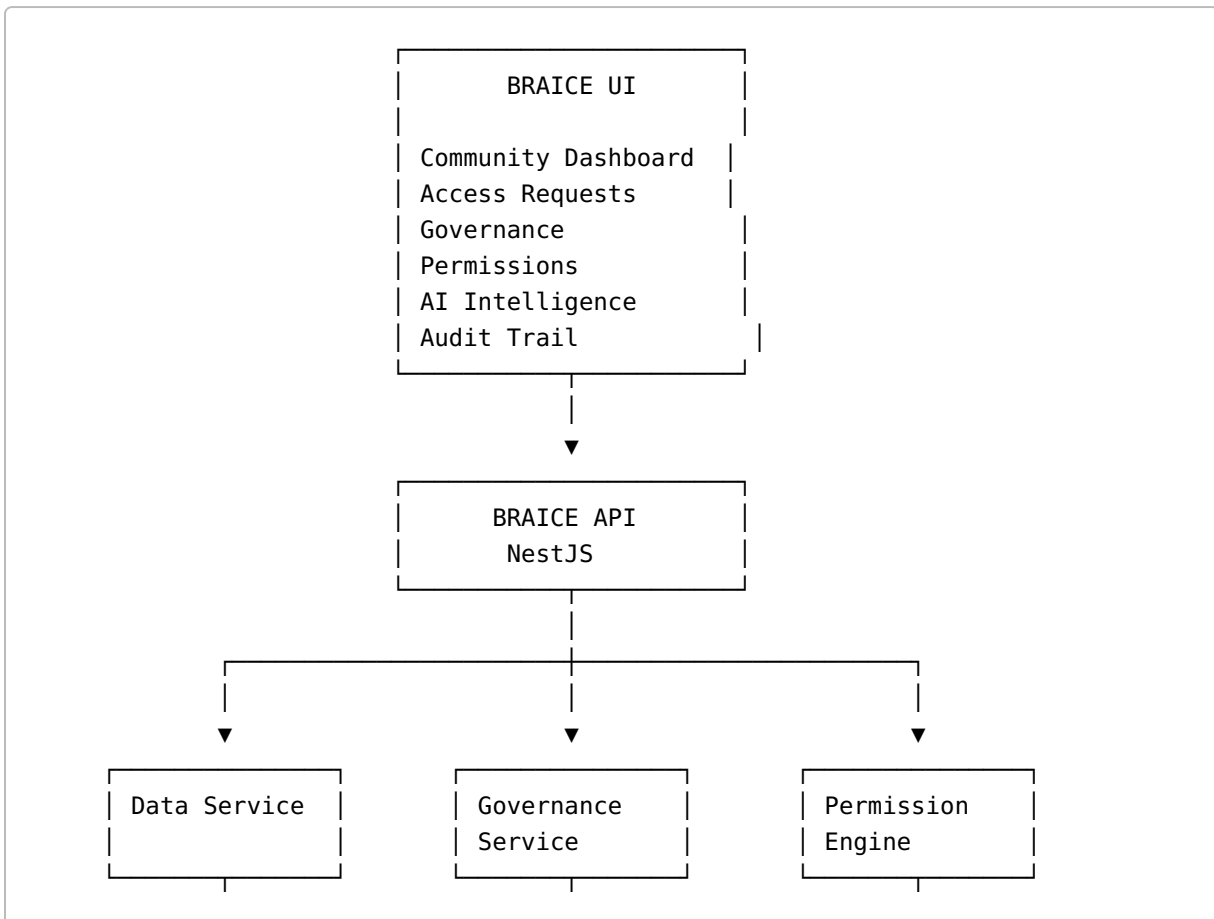
For the hackathon:

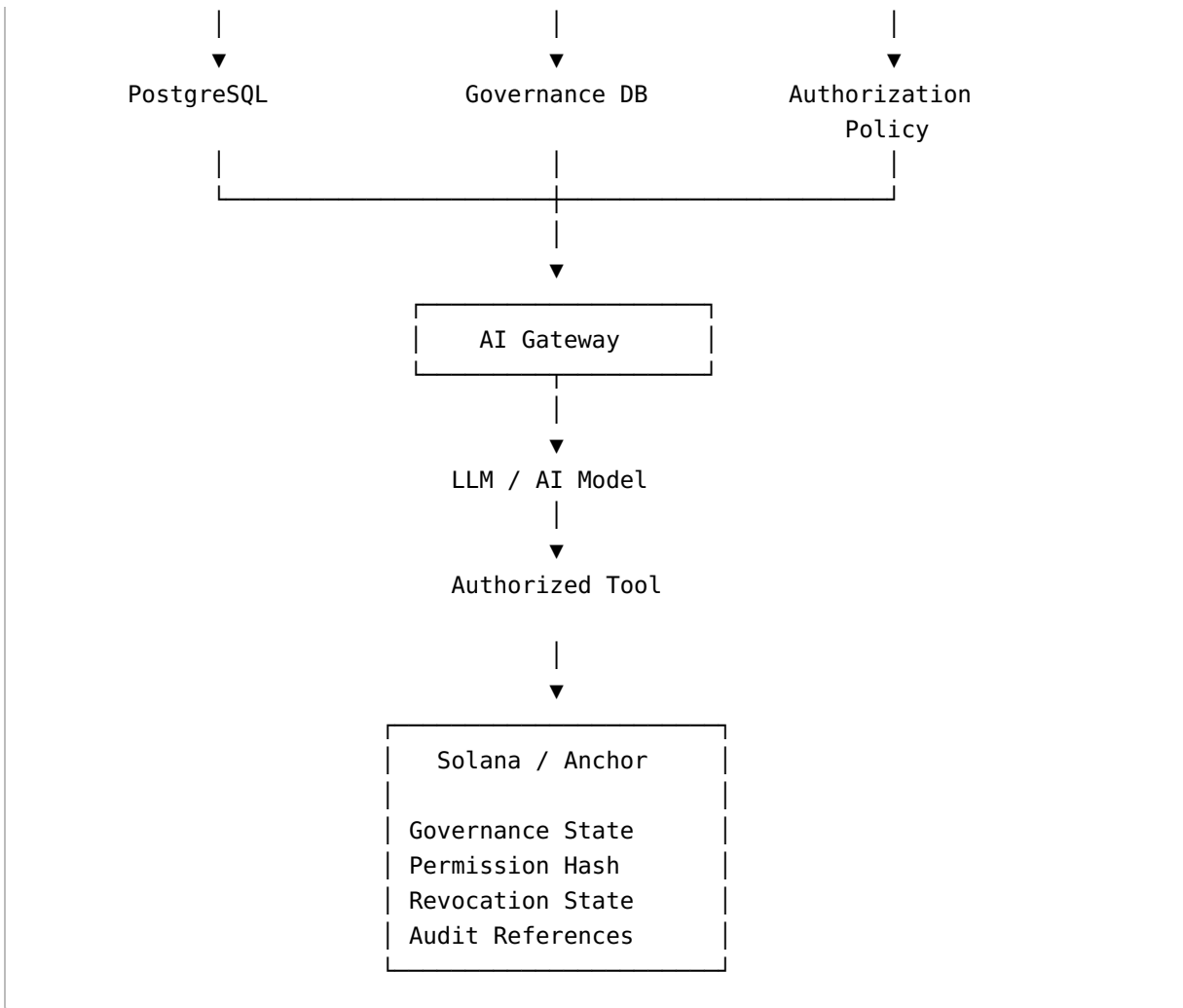
- Docker
- Docker Compose
- PostgreSQL
- Redis optional
- Next.js
- NestJS
- Solana devnet
- Anchor
- GitHub Actions

Avoid Kubernetes for this prototype.

Kubernetes would add operational complexity without improving the core proof.

6. High-Level System Architecture





7. Core Domain Model

BRAICE should not be modeled simply as:

User → Role → Permission

That would reduce BRAICE to a conventional RBAC system.

Instead:

Identity
↓
Relationship / Rights

↓
Governance
↓
Permission
↓
Resource
↓
Application

This follows the conceptual model in the product specification.

8. Database Schema

8.1 users

```
CREATE TABLE users (  
  id UUID PRIMARY KEY,  
  wallet_address TEXT UNIQUE,  
  email TEXT,  
  display_name TEXT NOT NULL,  
  user_type TEXT NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Possible user types:

INDIVIDUAL
CREATOR
BRAND
APPLICATION
ADMIN

For the hackathon we only need:

- CREATOR
 - MEMBER
 - BRAND
 - APPLICATION
-

9. communities

```
CREATE TABLE communities (  
  id UUID PRIMARY KEY,  
  name TEXT NOT NULL,  
  description TEXT,  
  operator_id UUID NOT NULL REFERENCES users(id),  
  governance_config JSONB NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Example:

```
{  
  "approval_mode": "CREATOR_AND_THRESHOLD",  
  "threshold_percentage": 60  
}
```

10. memberships

```
CREATE TABLE memberships (  
  id UUID PRIMARY KEY,  
  community_id UUID NOT NULL REFERENCES communities(id),  
  user_id UUID NOT NULL REFERENCES users(id),  
  role TEXT NOT NULL,  
  status TEXT NOT NULL,  
  joined_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  UNIQUE(community_id, user_id)  
);
```

Roles:

```
OPERATOR  
MEMBER
```

11. activity_records

This table represents individual-level activity.

```
CREATE TABLE activity_records (  
  id UUID PRIMARY KEY,  
  community_id UUID NOT NULL REFERENCES communities(id),  
  member_id UUID NOT NULL REFERENCES users(id),  
  activity_type TEXT NOT NULL,  
  interest_category TEXT NOT NULL,  
  metadata JSONB,  
  occurred_at TIMESTAMPTZ NOT NULL  
);
```

Example:

```
member_001  
clicked  
streetwear  
2026-09-13
```

This table is highly sensitive in the architecture.

The brand and AI application must never directly query it.

12. community_datasets

The aggregation service generates a community-level dataset.

```
CREATE TABLE community_datasets (  
  id UUID PRIMARY KEY,  
  community_id UUID NOT NULL REFERENCES communities(id),  
  dataset_type TEXT NOT NULL,  
  version INTEGER NOT NULL,  
  data JSONB NOT NULL,  
  source_count INTEGER NOT NULL,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Example:

```
{
  "streetwear": 42,
  "music_festivals": 31,
  "sneakers": 27,
  "beauty": 18
}
```

Notice that the dataset does not contain:

```
member_id
name
email
phone
wallet
individual activity
```

13. access_requests

```
CREATE TABLE access_requests (
  id UUID PRIMARY KEY,
  community_id UUID NOT NULL REFERENCES communities(id),
  requester_id UUID NOT NULL REFERENCES users(id),
  dataset_id UUID NOT NULL REFERENCES community_datasets(id),
  purpose TEXT NOT NULL,
  operation TEXT NOT NULL,
  requested_duration_seconds INTEGER NOT NULL,
  status TEXT NOT NULL,
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);
```

Example:

```
{
  "requester": "Nike",
  "dataset": "Community Interests",
  "purpose": "Campaign Planning",
  "operation": "ANALYZE",
  "duration": "30 days"
}
```

14. governance_decisions

```
CREATE TABLE governance_decisions (  
  id UUID PRIMARY KEY,  
  access_request_id UUID NOT NULL REFERENCES access_requests(id),  
  decision TEXT NOT NULL,  
  approved_by JSONB NOT NULL,  
  approval_count INTEGER,  
  threshold INTEGER,  
  decided_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),  
  blockchain_tx TEXT  
);
```

Possible decisions:

```
APPROVED  
REJECTED
```

15. permissions

This is the central BRAICE object.

```
CREATE TABLE permissions (  
  id UUID PRIMARY KEY,  
  access_request_id UUID NOT NULL REFERENCES access_requests(id),  
  principal_id UUID NOT NULL REFERENCES users(id),  
  resource_id UUID NOT NULL REFERENCES community_datasets(id),  
  purpose TEXT NOT NULL,  
  operation TEXT NOT NULL,  
  conditions JSONB,  
  issued_at TIMESTAMPTZ NOT NULL,  
  expires_at TIMESTAMPTZ NOT NULL,  
  revoked_at TIMESTAMPTZ,  
  status TEXT NOT NULL,  
  policy_hash TEXT,  
  blockchain_reference TEXT  
);
```

A permission answers:

```
WHO?  
WHAT?  
WHY?  
HOW?  
FOR HOW LONG?  
UNDER WHAT CONDITIONS?
```

That corresponds directly to the permission model defined in the BRAICE specification.

16. audit_events

```
CREATE TABLE audit_events (  
  id UUID PRIMARY KEY,  
  community_id UUID REFERENCES communities(id),  
  actor_id UUID REFERENCES users(id),  
  event_type TEXT NOT NULL,  
  resource_id UUID,  
  permission_id UUID,  
  metadata JSONB,  
  blockchain_tx TEXT,  
  created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()  
);
```

Examples:

```
ACCESS_REQUESTED  
GOVERNANCE_APPROVED  
PERMISSION_CREATED  
AI_ACCESS_GRANTED  
AI_ANALYSIS_COMPLETED  
PERMISSION_REVOKED  
ACCESS_DENIED
```

17. Permission Object

Internally, BRAICE should represent permissions approximately like this:

```
interface Permission {  
  id: string;
```

```
principal: {
  type: "USER" | "APPLICATION";
  id: string;
};

resource: {
  type: "COMMUNITY_DATASET";
  id: string;
};

purpose: string;

operations: (
  | "READ"
  | "ANALYZE"
)[];

conditions: {
  aggregationLevel: "COMMUNITY";
  allowIndividualData: false;
};

issuedAt: Date;
expiresAt: Date;

status:
  | "PENDING"
  | "ACTIVE"
  | "EXPIRED"
  | "REVOKED";
}
```

The important property is:

```
allowIndividualData: false
```

But this must not merely be a UI flag.

The backend must technically enforce it.

18. Permission Engine

The permission engine is the most important component of the prototype.

Create:

```
PermissionService
```

with:

```
checkAccess(request): AuthorizationDecision
```

Conceptually:

```
checkAccess({  
  principal,  
  resource,  
  purpose,  
  operation  
})
```

returns:

```
{  
  allowed: true,  
  permissionId: "...",  
  reason: "ACTIVE_PERMISSION"  
}
```

or:

```
{  
  allowed: false,  
  reason: "PERMISSION_REVOKED"  
}
```

19. Permission Evaluation Algorithm

Every protected request follows:

1. Identify principal
2. Identify resource

3. Identify requested operation
4. Find applicable permission
5. Verify permission status
6. Verify expiration
7. Verify purpose
8. Verify operation
9. Verify resource scope
10. Verify conditions
11. ALLOW or DENY

Pseudo-code:

```
async function authorize(input: AccessRequest) {

  const permission =
    await permissionRepository.findActivePermission(
      input.principalId,
      input.resourceId
    );

  if (!permission) {
    return deny("NO_PERMISSION");
  }

  if (permission.status !== "ACTIVE") {
    return deny("PERMISSION_NOT_ACTIVE");
  }

  if (permission.expiresAt < new Date()) {
    return deny("PERMISSION_EXPIRED");
  }

  if (!permission.operations.includes(input.operation)) {
    return deny("OPERATION_NOT_ALLOWED");
  }

  if (permission.purpose !== input.purpose) {
    return deny("PURPOSE_MISMATCH");
  }

  if (input.requestsIndividualData &&
    !permission.conditions.allowIndividualData) {
    return deny("INDIVIDUAL_DATA_RESTRICTED");
  }
}
```

```
    return allow(permission.id);  
}
```

This is the heart of the BRAICE prototype.

20. Governance Engine

The governance engine converts a request into a decision.

For the hackathon, keep the governance rule deliberately simple.

Example:

```
Creator approval  
+  
60% community approval  
=  
APPROVED
```

The governance service receives:

```
evaluateGovernance(accessRequest)
```

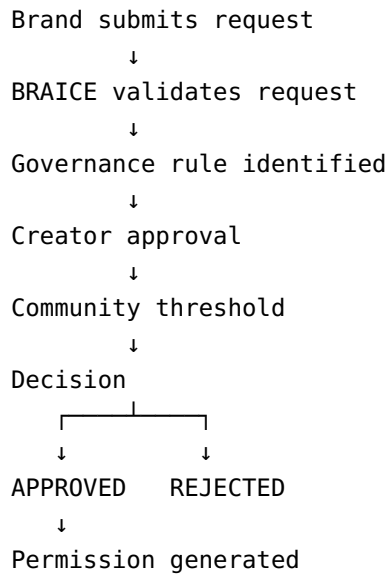
and produces:

```
{  
  decision: "APPROVED",  
  reason: "CREATOR_APPROVED_AND_THRESHOLD_MET"  
}
```

The important thing is that the resulting decision creates an actual permission.

The product concept explicitly says governance becomes operational when a governance decision results in a concrete authorization state that applications can enforce.

21. Governance Flow



For the hackathon, the community vote can be simulated through seeded members or a simple approval interface.

Do not spend days building a sophisticated voting protocol.

22. Aggregation Pipeline

The aggregation pipeline must demonstrate the separation between individual data and community intelligence.

Input:

100 individual records

Example:

```
member_001 → sneakers
member_002 → streetwear
member_003 → sneakers
member_004 → music
...
```

Aggregation:

```
GROUP BY interest_category  
COUNT(*)
```

Output:

Streetwear	42%
Music	31%
Sneakers	27%
Beauty	18%

The output becomes a separate `community_dataset`.

The commercial partner receives this dataset—not the underlying member records.

This separation is central to the BRAICE concept.

23. The Critical Security Boundary

This must be explicitly enforced.

The following must be impossible:

```
Brand → /api/activity-records
```

and:

```
AI → PostgreSQL → activity_records
```

Instead:

```
AI  
↓  
AI Gateway  
↓  
Permission Engine  
↓  
Authorized Dataset Service
```

```
↓  
community_datasets
```

The AI should have no database credentials.

The AI should have no arbitrary SQL tool.

The AI should have no generic database query tool.

24. AI Agent Architecture

The AI agent should have only a small set of tools.

For the hackathon:

```
get_community_insight()
```

and optionally:

```
analyze_authorized_dataset()
```

The agent asks BRAICE:

```
Can I access dataset X?
```

BRAICE evaluates the permission.

If allowed:

```
return authorized dataset
```

If denied:

```
return authorization error
```

25. AI Flow

User:
"What are the strongest emerging interests?"

↓

AI Agent

↓

Requests:
community dataset

↓

Permission Engine

↓

Is AI authorized?

↓

YES

↓

Authorized Dataset

↓

LLM

↓

Insight

The AI response might be:

Streetwear is currently the strongest emerging interest, followed by music festivals and sneakers.

26. The Attack / Restriction Demo

This is essential.

After the successful AI analysis, ask:

"Show me the individual members who are interested in streetwear."

The AI attempts to perform the request.

BRAICE responds:

ACCESS DENIED

Reason:

The current permission authorizes analysis of community-level aggregated data only.

Individual-level records are outside the authorized resource scope.

This proves that the permission layer is controlling the AI rather than merely instructing the AI to behave.

27. Revocation Architecture

Revocation must happen server-side.

When the creator clicks:

REVOKE ACCESS

BRAICE changes:

ACTIVE

to:

REVOKED

Every subsequent authorization check sees:

status = REVOKED

and rejects the request.

28. Revocation Flow

```
Creator
  ↓
Revoke Permission
  ↓
Permission Service
  ↓
status = REVOKED
  ↓
Audit Event
  ↓
Blockchain Event
```

Then:

```
Brand / AI
  ↓
Requests dataset
  ↓
Permission Engine
  ↓
REVOKED
  ↓
ACCESS DENIED
```

This must be tested automatically.

29. Solana Smart Contract Architecture

The blockchain program should be deliberately small.

Do not attempt to put BRAICE's entire authorization system on-chain.

Use Solana as a **verifiable state layer**.

The product concept explicitly identifies governance decisions, permission states, authorization events, revocations, credentials, proofs and audit history as candidates for verifiable state while keeping sensitive information off-chain.

30. Proposed Anchor Program

Program:

```
braice_governance
```

Accounts:

```
CommunityState  
PermissionState  
GovernanceEvent
```

31. CommunityState

Conceptual Anchor account:

```
pub struct CommunityState {  
    pub community_id: [u8; 32],  
    pub operator: Pubkey,  
    pub governance_rule_hash: [u8; 32],  
    pub created_at: i64,  
    pub bump: u8,  
}
```

Do NOT store:

```
member names  
emails  
activity records  
AI data  
personal information
```

32. PermissionState

```
pub struct PermissionState {  
    pub permission_id: [u8; 32],
```

```
pub community_id: [u8; 32],
pub requester: Pubkey,

pub dataset_hash: [u8; 32],
pub purpose_hash: [u8; 32],

pub issued_at: i64,
pub expires_at: i64,

pub status: u8,

pub policy_hash: [u8; 32],

pub bump: u8,
}
```

Status:

```
0 = ACTIVE
1 = REVOKED
2 = EXPIRED
```

33. Smart Contract Instructions

Keep the program to approximately four core instructions.

initialize_community

Creates the verifiable community state.

record_governance_decision

Records:

```
community
request
decision
timestamp
decision hash
```

create_permission

Creates a permission state derived from an approved governance decision.

revoke_permission

Changes:

ACTIVE → REVOKED

and emits an event.

34. Blockchain Is Not the Authorization Engine

This distinction is important.

Do NOT implement:

```
API request
↓
wait for blockchain transaction
↓
check permission
```

for every request.

That would unnecessarily couple runtime authorization to blockchain latency.

Instead:

```
Governance
↓
Permission created
↓
Verifiable blockchain state

Runtime request
↓
BRAICE Permission Engine
```

↓
Fast authorization

The blockchain provides verifiable state/proof.

The backend provides low-latency enforcement.

35. Permission Hash

When creating a permission, canonicalize:

```
{
  "principal": "...",
  "resource": "...",
  "purpose": "...",
  "operations": ["ANALYZE"],
  "expiresAt": "...",
  "conditions": {
    "aggregationLevel": "COMMUNITY",
    "allowIndividualData": false
  }
}
```

Hash the canonical representation.

Example:

SHA-256(permission_json)

Store:

policy_hash

on-chain.

This lets the demo show:

BRAICE Database
↓
Permission

↓
Policy Hash
↓
Solana
↓
Verifiable State

36. API Architecture

The backend should expose domain-oriented APIs.

Community

```
POST /api/communities
GET  /api/communities/:id
GET  /api/communities/:id/members
```

Activity

For internal/demo ingestion:

```
POST /api/communities/:id/activity
```

This endpoint must never be available to the brand role.

Intelligence

```
POST /api/communities/:id/datasets/generate
GET  /api/datasets/:id
```

Access Requests

```
POST /api/access-requests
GET /api/access-requests/:id
GET /api/communities/:id/access-requests
```

Request:

```
{
  "communityId": "...",
  "datasetId": "...",
  "purpose": "campaign_planning",
  "operation": "ANALYZE",
  "durationSeconds": 2592000
}
```

37. Governance APIs

```
POST /api/access-requests/:id/governance/approve
POST /api/access-requests/:id/governance/reject
GET /api/access-requests/:id/governance
```

For the hackathon, these can represent the creator and simulated member approvals.

38. Permission APIs

```
GET /api/permissions/:id
POST /api/permissions/:id/revoke
POST /api/authorize
```

The important endpoint is:

```
POST /api/authorize
```

Example:

```
{
  "principalId": "ai-agent",
  "resourceId": "community-dataset-123",
  "purpose": "campaign_planning",
  "operation": "ANALYZE"
}
```

Response:

```
{
  "allowed": true,
  "permissionId": "perm-123"
}
```

After revocation:

```
{
  "allowed": false,
  "reason": "PERMISSION_REVOKED"
}
```

39. AI API

Do not expose the LLM directly.

Use:

```
POST /api/ai/query
```

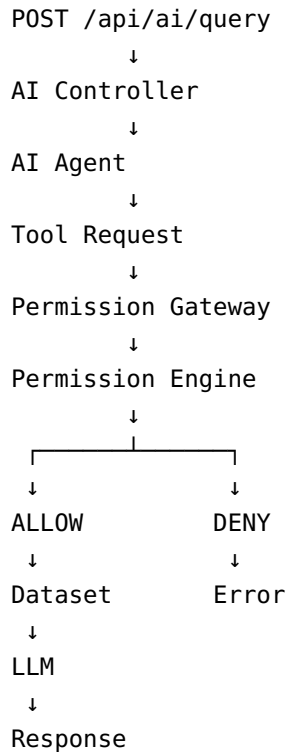
Request:

```
{
  "communityId": "...",
  "question": "What are the strongest emerging interests?"
}
```

The AI service determines what data it needs.

It then goes through the permission gateway.

40. AI Authorization Pipeline



41. Audit API

`GET /api/communities/:id/audit`
`GET /api/permissions/:id/audit`

The audit page should show:

```
11:02:14 ACCESS_REQUESTED
11:03:01 GOVERNANCE_APPROVED
11:03:05 PERMISSION_CREATED
11:04:10 AI_ACCESS_GRANTED
11:04:13 AI_ANALYSIS_COMPLETED
11:05:21 PERMISSION_REVOKED
11:05:30 ACCESS_ATTEMPT
11:05:30 ACCESS_DENIED
```

42. Repository Structure

Use a monorepo.

```
braice/
├── apps/
│   ├── web/
│   │   ├── app/
│   │   ├── components/
│   │   ├── lib/
│   │   └── hooks/
│   └── api/
│       ├── src/
│       │   ├── modules/
│       │   │   ├── communities/
│       │   │   ├── memberships/
│       │   │   ├── activity/
│       │   │   ├── datasets/
│       │   │   ├── access-requests/
│       │   │   ├── governance/
│       │   │   ├── permissions/
│       │   │   ├── authorization/
│       │   │   ├── ai/
│       │   │   ├── audit/
│       │   │   └── blockchain/
│       │   ├── common/
│       │   ├── config/
│       │   └── main.ts
│       └── test/
├── packages/
│   ├── types/
│   ├── permission-engine/
│   ├── governance-engine/
│   ├── blockchain-client/
│   └── ai-client/
├── programs/
│   └── braice-governance/
│       ├── programs/
│       └── tests/
```

```

├── Anchor.toml
├── database/
│   ├── migrations/
│   ├── seeds/
│   └── fixtures/
├── scripts/
│   ├── seed-demo.ts
│   ├── generate-activity.ts
│   └── deploy-devnet.ts
├── docs/
│   ├── architecture.md
│   ├── permission-model.md
│   ├── governance-model.md
│   └── demo-script.md
├── docker-compose.yml
├── package.json
├── pnpm-workspace.yaml
└── README.md

```

43. Recommended Module Ownership

If the team has three engineers:

Engineer 1 — Core Protocol

Own:

```

permission-engine
governance-engine
database model
authorization tests

```

This is the highest-priority engineering role.

Engineer 2 — Solana + AI

Own:

```
Anchor program
Solana client
AI gateway
AI tools
policy hashing
blockchain event recording
```

Engineer 3 — Product/UI

Own:

```
Next.js
community dashboard
access request UI
governance UI
permission UI
AI interface
audit trail
demo experience
```

Everyone should understand the permission engine.

44. Frontend Screens

You only need approximately six screens.

Screen 1 — Community Dashboard

Show:

```
Afrobeat Creators

Members: 100

Community Intelligence
-----
Streetwear      42%
Music Festivals 31%
```

Sneakers	27%
Beauty	18%

Screen 2 — Brand Access Request

Nike

Request Access

Dataset:
Community Interests

Purpose:
Campaign Planning

Operation:
AI Analysis

Duration:
30 days

[Request Access]

Screen 3 — Governance

Show:

Access Request

Nike wants:
Community-level interest intelligence

Purpose:
Campaign Planning

Creator Approval:
✓ Approved

Community Approval:
63 / 100

Required:
60

STATUS:
APPROVED

Screen 4 — Permission

Show the actual authorization object:

BRAICE PERMISSION

Principal:
Nike AI

Resource:
Community Interest Dataset

Purpose:
Campaign Planning

Operation:
Analyze

Data Level:
Community Aggregated

Individual Data:
BLOCKED

Expires:
30 days

Status:
ACTIVE

Policy Hash:
8a91...e32

Screen 5 — AI

Chat interface:

BRAICE Intelligence Agent

Question:

What are the strongest emerging interests?

Answer:

Streetwear is the strongest emerging interest...

Then:

Question:

Show individual members interested in streetwear.

BRAICE:

ACCESS DENIED.

The current permission authorizes analysis of community-level aggregated data only.

Screen 6 — Revocation

Show:

Permission #8472

STATUS:

ACTIVE

[REVOKE ACCESS]

After clicking:

STATUS:

REVOKED

Then repeat the AI request:

ACCESS DENIED

Reason :
PERMISSION_REVOKED

45. The End-to-End Demo State Machine

The demo should have deterministic state.

```
REQUESTED
  ↓
GOVERNANCE_PENDING
  ↓
APPROVED
  ↓
PERMISSION_ACTIVE
  ↓
AI_AUTHORIZED
  ↓
ANALYSIS_COMPLETED
  ↓
REVOKED
  ↓
ACCESS_DENIED
```

This state machine should be represented explicitly in the backend.

Do not rely on frontend state.

46. Demo Data

Use synthetic data.

Generate 1,000 activity records across approximately:

```
Streetwear
Music
Sneakers
Beauty
Gaming
Events
```

```
Technology
Travel
```

The distribution should intentionally create obvious trends.

Example:

```
Streetwear      42%
Music Festivals 31%
Sneakers        27%
Beauty          18%
Gaming          15%
Technology       12%
Travel          9%
```

The exact numbers do not matter.

What matters is that the AI can produce a meaningful community-level insight.

47. Security Tests

Security testing should be one of the strongest parts of the submission.

Test 1 — Unauthorized brand access

Brand attempts:

```
GET individual activity records
```

Expected:

```
403
```

Test 2 — AI individual-data request

AI requests:

member records

Expected:

DENIED

Test 3 — Wrong purpose

Permission:

campaign_planning

Request:

advertising_targeting

Expected:

DENIED

Test 4 — Expired permission

Permission expiration:

2026-09-01

Request:

2026-09-13

Expected:

DENIED

Test 5 — Revoked permission

Expected:

DENIED

Test 6 — Wrong operation

Permission:

ANALYZE

Request:

EXPORT

Expected:

DENIED

48. Required MVP Acceptance Tests

The tests should directly map to the EXIST MVP success criteria.

Data separation

100% of tested commercial/AI requests return only the authorized aggregated dataset.

The MVP specification identifies exposure of even one unauthorized individual-level record as failure.

Governance

At least 95% of defined governance scenarios produce the expected decision.

Permission enforcement

100% of unauthorized access attempts are denied.

Revocation

100% of revoked permissions prevent subsequent access.

AI restriction

100% of tested attempts by AI to access restricted individual-level data are blocked.

Audit

Governance, authorization and revocation events must be verifiable.

49. Day-by-Day Build Plan

Assuming approximately 10–14 days of hackathon development.

DAY 1 — Architecture + Repository

Deliver:

```
Monorepo
Next.js app
NestJS app
PostgreSQL
Anchor project
Docker setup
CI
```

Also freeze:

```
Permission model
Governance model
Data model
API contracts
```

Do not start UI polish.

DAY 2 — Database + Seed Data

Implement:

```
users
communities
memberships
activity_records
community_datasets
```

Create seed script.

Generate:

```
100 members
1,000 activity records
```

Verify aggregation.

Deliverable:

```
Individual data
    ↓
Aggregation
    ↓
Community dataset
```

DAY 3 — Governance

Implement:

```
Access Request
Governance Rule
Governance Decision
```

Build:

Creator approval
Community threshold

Test:

APPROVED
REJECTED

DAY 4 — Permission Engine

This is the most important day.

Implement:

```
createPermission()  
checkPermission()  
revokePermission()  
expirePermission()
```

Implement checks for:

```
principal  
resource  
purpose  
operation  
expiration  
status  
conditions  
data level
```

Write unit tests immediately.

DAY 5 — Protected Data Gateway

Create:

```
AuthorizedDatasetService
```

The AI and brand APIs must use this service.

Remove direct access to protected data.

Architecture must become:

```
Client
↓
API
↓
Authorization
↓
Authorized Data Service
↓
Dataset
```

DAY 6 — AI Integration

Implement:

```
POST /api/ai/query
```

Create one AI tool:

```
analyzeCommunityDataset
```

The tool must call:

```
Permission Engine
```

before receiving any dataset.

DAY 7 — Solana Program

Implement:

```
initialize_community  
record_governance_decision  
create_permission  
revoke_permission
```

Deploy to Solana devnet.

Store transaction signatures.

DAY 8 — Blockchain Integration

When important events happen:

```
Governance Approved  
Permission Created  
Permission Revoked
```

write corresponding verifiable blockchain state.

Store:

```
transaction signature
```

in PostgreSQL.

Create a verification page/link inside the demo.

DAY 9 — Frontend

Build the six screens:

```
Community  
Access Request  
Governance  
Permission  
AI  
Revocation/Audit
```

Do not spend time on complex navigation.

The entire demo should feel like one workflow.

DAY 10 — Full Integration

Run:

```
Community
↓
Data
↓
Aggregation
↓
Brand Request
↓
Governance
↓
Permission
↓
AI
↓
Revocation
↓
Denied
```

Fix integration issues.

DAY 11 — Security Testing

Attack your own system.

Try:

```
Brand → individual data
AI → individual data
AI → expired permission
AI → revoked permission
Brand → wrong purpose
```

Brand → wrong operation
Brand → wrong dataset

Everything unauthorized must fail.

DAY 12 — Blockchain Verification + Audit

Make the audit trail extremely clear.

Show:

```
Governance Decision
  ↓
Permission Hash
  ↓
Solana Transaction
  ↓
Permission Active
  ↓
Revocation
```

DAY 13 — Demo Hardening

Remove unnecessary features.

Seed deterministic data.

Ensure every demo action works.

Prepare fallback local/demo mode if devnet/network problems occur.

DAY 14 — Pitch + Recording

Record the complete demonstration.

The recorded demo should work without improvisation.

50. The Exact Demo Script

Scene 1 — Problem

Show:

Afrobeat Creators
100 members

Explain:

This community generates valuable behavioral information, but the community should be able to govern how that information is used.

Scene 2 — Data

Show:

1,000 individual activities

Then:

Community Intelligence

Streetwear 42%
Music 31%
Sneakers 27%

Explain:

BRAICE separates the individual records from the community-level intelligence derived from them.

Scene 3 — Brand Request

Nike requests:

Community Intelligence
Purpose: Campaign Planning
Operation: Analyze
Duration: 30 days

Scene 4 — Governance

Creator approves.

Community threshold passes.

BRAICE records:

GOVERNANCE APPROVED

Scene 5 — Permission

Show the permission.

This is where you explain:

Governance did not merely produce a database flag. BRAICE translated the decision into a machine-readable authorization policy.

Scene 6 — AI

Ask:

What are the strongest emerging interests?

AI answers.

Then ask:

Show me individual members interested in streetwear.

BRAICE:

ACCESS DENIED

This is the critical demonstration.

Scene 7 — Revocation

Click:

REVOKE

Permission becomes:

REVOKED

Ask AI the original question again.

Result:

ACCESS DENIED

Scene 8 — Verification

Show the audit trail and Solana transaction.

Explain:

The sensitive data remains off-chain. Solana provides a verifiable record of the governance and authorization state.

51. What Makes This Different From RBAC

A judge may ask:

"Isn't this just role-based access control?"

The answer is:

No.

Traditional RBAC:

```
User
↓
Role
↓
Permission
```

BRAICE:

```
Identity
↓
Rights / Relationships
↓
Governance
↓
Purpose
↓
Permission
↓
Conditions
↓
Resource
↓
Application / AI
```

The permission is dynamic.

It can be:

```
created
approved
modified
expired
revoked
```

The BRAICE product specification explicitly identifies this dynamic authorization lifecycle as one of the core technological differentiators.

52. What Makes This Different From a DAO

A judge may ask:

"Why isn't this just a DAO?"

Because governance is only one layer.

BRAICE is not trying to make every decision a token vote.

Governance can be:

```
creator approval  
representative governance  
delegated authority  
predefined rules  
thresholds  
stakeholder combinations  
automated conditions
```

The result is a permission that applications can enforce.

That governance-to-enforcement connection is the important part.

53. What Makes This Different From a Blockchain Database

Another likely question:

"Why do you need blockchain?"

The answer should be:

We don't use blockchain as our database.

Sensitive data remains off-chain.

Blockchain is used for verifiable shared state where independent verification has value.

For example:

Governance Decision
Permission State
Revocation
Proof

This is consistent with the product architecture's explicit separation of off-chain data from verifiable state.

54. What Makes This Different From an AI Data Platform

Another likely question:

"Isn't this just an AI analytics product?"

No.

AI is an application.

The actual BRAICE primitive is:

Governance
↓
Permission
↓
Enforcement

AI simply provides a compelling demonstration of why permission-aware applications matter.

The product concept explicitly states that AI is an application of the permission architecture rather than the primary innovation thesis.

55. The Most Important Engineering Rule

The team should adopt this rule:

Never trust the client to enforce permissions.

Bad:

```
Frontend:
if (permission) {
  showData();
}
```

Good:

```
Frontend
  ↓
Backend
  ↓
Permission Engine
  ↓
Authorized Data
```

Even if somebody bypasses the UI and directly calls the API, the API must reject unauthorized access.

56. Second Engineering Rule

The AI must not have unrestricted data access.

Never give the LLM:

```
DATABASE_URL
```

Never give it:

```
SQL access
```

Never give it:

```
activity_records
```

Give it only:

```
BRAICE tools
```

Those tools are permission-aware.

57. Third Engineering Rule

Blockchain should verify important state, not become the application database.

The architecture should remain:

```
PostgreSQL
  ↓
Application State

Solana
  ↓
Verifiable Governance / Permission State
```

58. Definition of Done

The hackathon prototype is DONE only when the following works from a clean environment.

Scenario A

Community generates activity.

PASS

Scenario B

BRAICE generates community intelligence.

PASS

Scenario C

Brand requests access.

PASS

Scenario D

Governance approves.

PASS

Scenario E

Permission is created.

PASS

Scenario F

AI analyzes authorized data.

PASS

Scenario G

AI attempts individual-level access.

DENIED

Scenario H

Permission is revoked.

PASS

Scenario I

AI tries again.

DENIED

Scenario J

Governance/permission/revocation state is verifiable.

PASS

If any of these fail, the prototype is not ready.

59. The Minimum Viable Architecture

If time becomes extremely constrained, reduce the system to:

```
Next.js
↓
NestJS
↓
PostgreSQL
↓
Permission Engine
↓
AI Gateway

+

Solana / Anchor
```

Everything else is secondary.

The permission engine must survive.

60. Priority Order

When the team has to choose between features, use this ranking:

P0 — Absolutely required

1. Permission engine
2. Governance → permission
3. Data separation
4. AI authorization
5. Revocation
6. Unauthorized access denial

P1 — Required for strong submission

1. Solana state
2. Audit trail
3. Community dashboard
4. Access request UI
5. Governance UI

P2 — Nice to have

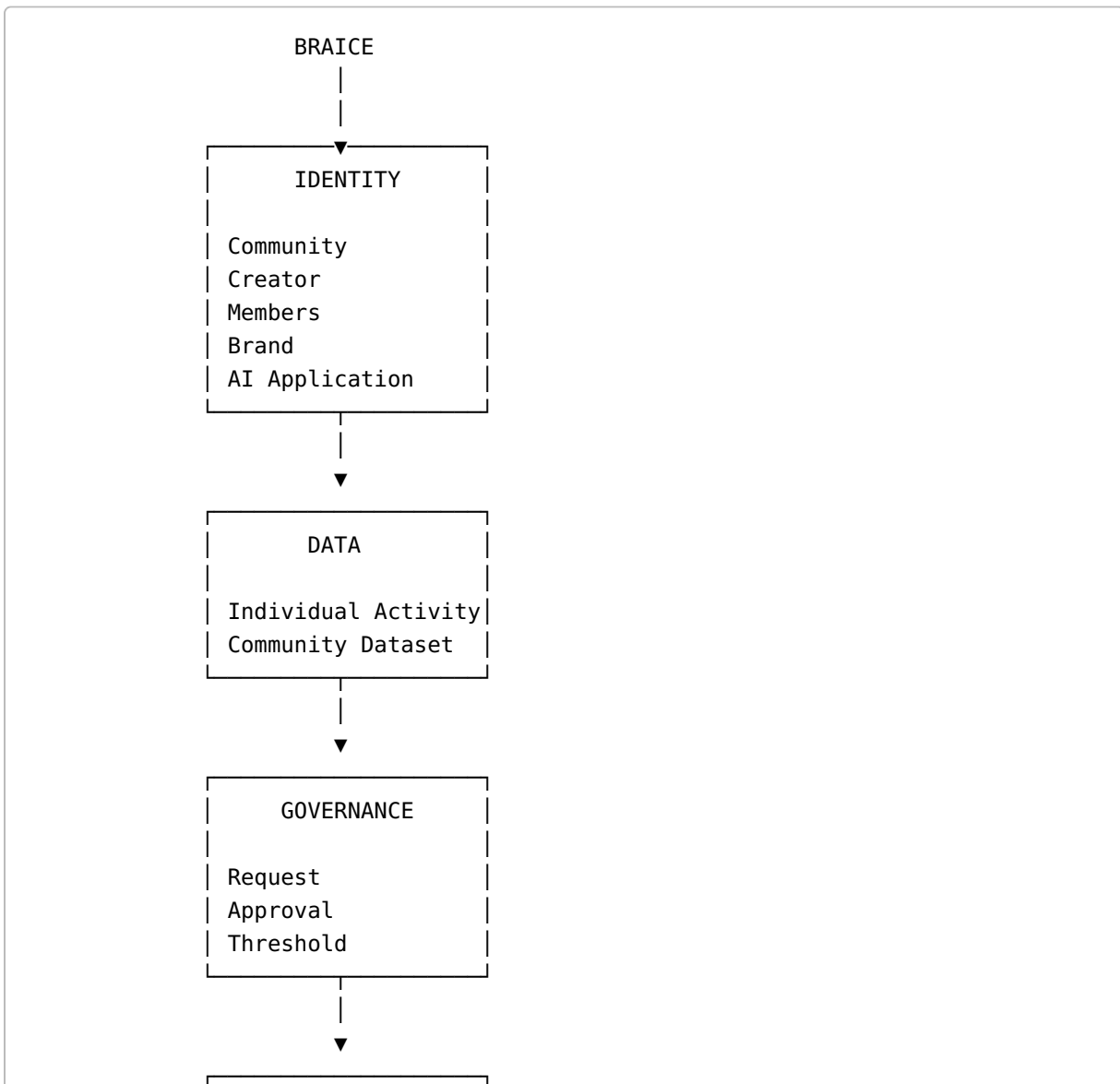
1. Better AI responses
2. Advanced analytics
3. Beautiful visualizations
4. Multiple governance rules

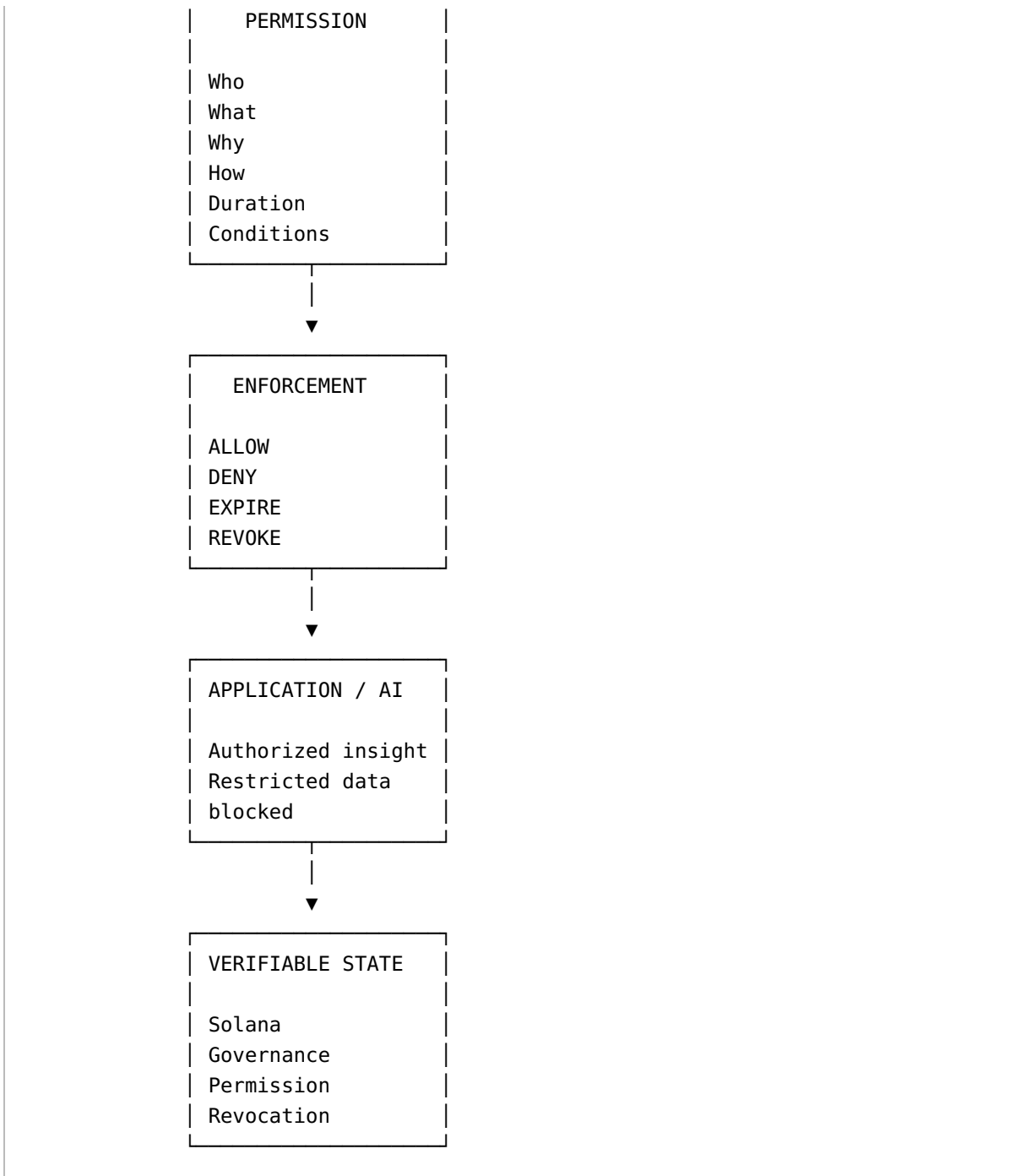
P3 — Do not build during hackathon

1. Tokens
2. NFT infrastructure
3. Marketplace
4. Multi-chain
5. Portable community identity
6. Full community platform

61. Final Product Architecture

The prototype should ultimately demonstrate:





62. The One Sentence the Engineering Team Should Remember

We are not building a community platform; we are building a working permission infrastructure that proves a community governance decision can control what an AI application is technically allowed to do with community data.

That is the Colosseum prototype.

The larger BRAICE vision can then grow from this primitive into portable community state and eventually community economic infrastructure, as described in the full product concept.